AD_DA底层

官方参考

官方会给我们一个底层用于参考（其实就是iic总线的通信底层，这个底层我们不用去更改任何东西）

```c
/*  #   I2C代码片段说明
    1.  本文件夹中提供的驱动代码供参赛选手完成程序设计参考。
    2.  参赛选手可以自行编写相关代码或以该代码为基础，根据所选单片机类型、运行速度和试题
        中对单片机时钟频率的要求，进行代码调试和修改。
*/

#define DELAY_TIME  10 //如果出现问题，可以修改为5

//
static void I2C_Delay(unsigned char n)
{
    do
    {
        _nop_();_nop_();_nop_();_nop_();_nop_();
        _nop_();_nop_();_nop_();_nop_();_nop_();
        _nop_();_nop_();_nop_();_nop_();_nop_();
    }
    while(n--);
}

//
void I2CStart(void)
{
    sda = 1;
    scl = 1;
    I2C_Delay(DELAY_TIME);
    sda = 0;
    I2C_Delay(DELAY_TIME);
    scl = 0;
}

//
void I2CStop(void)
{
    sda = 0;
```

```
    scl = 1;
    I2C_Delay(DELAY_TIME);
    sda = 1;
    I2C_Delay(DELAY_TIME);
}

//
void I2CSendByte(unsigned char byt)
{
    unsigned char i;

    for(i=0; i<8; i++){
        scl = 0;
        I2C_Delay(DELAY_TIME);
        if(byt & 0x80){
            sda = 1;
        }
        else{
            sda = 0;
        }
        I2C_Delay(DELAY_TIME);
        scl = 1;
        byt <<= 1;
        I2C_Delay(DELAY_TIME);
    }

    scl = 0;
}

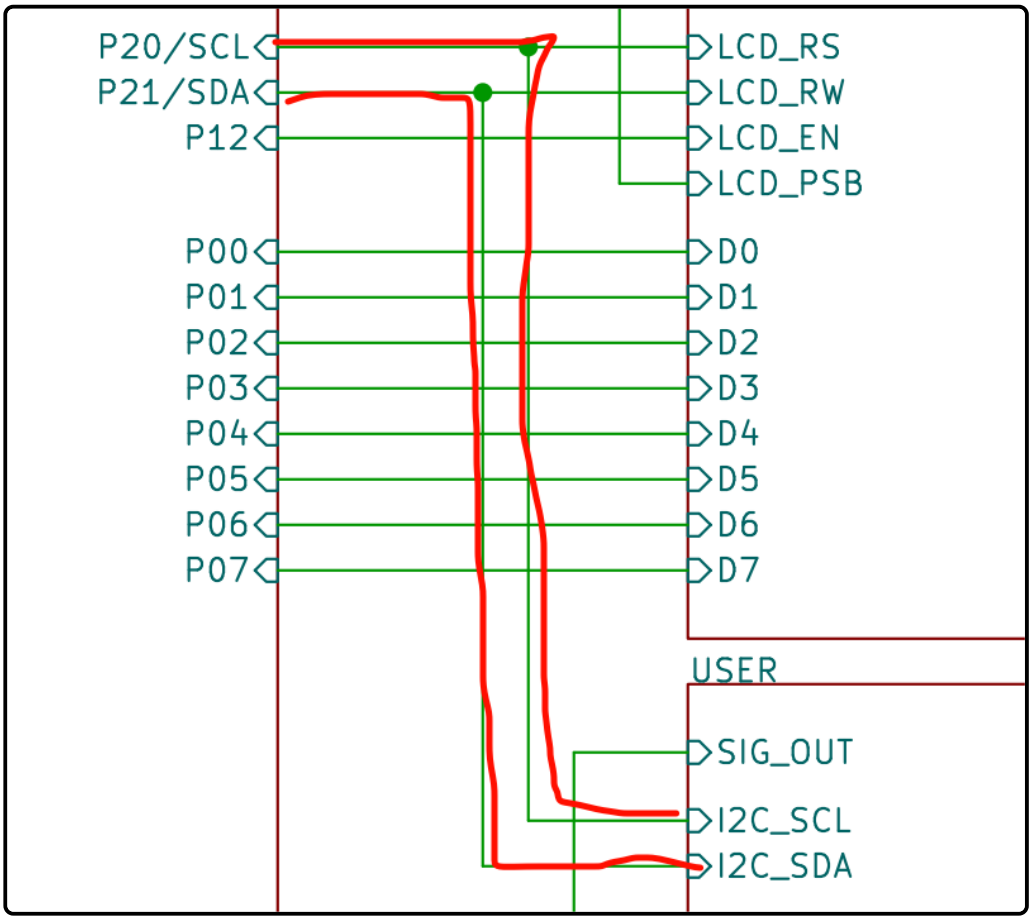
//
unsigned char I2CReceiveByte(void)
{
    unsigned char da;
    unsigned char i;
    for(i=0;i<8;i++){
        scl = 1;
        I2C_Delay(DELAY_TIME);
        da <<= 1;
        if(sda)
            da |= 0x01;
        scl = 0;
        I2C_Delay(DELAY_TIME);
    }
    return da;
}

//
unsigned char I2CWaitAck(void)
{
    unsigned char ackbit;

```

```
87      scl = 1;
88      I2C_Delay(DELAY_TIME);
89      ackbit = sda;
90      scl = 0;
91      I2C_Delay(DELAY_TIME);
92
93      return ackbit;
94  }
95
96  //
97  void I2CSendAck(unsigned char ackbit)
98  {
99      scl = 0;
100     sda = ackbit;
101     I2C_Delay(DELAY_TIME);
102     scl = 1;
103     I2C_Delay(DELAY_TIME);
104     scl = 0;
105     sda = 1;
106     I2C_Delay(DELAY_TIME);
107 }
108
109
```

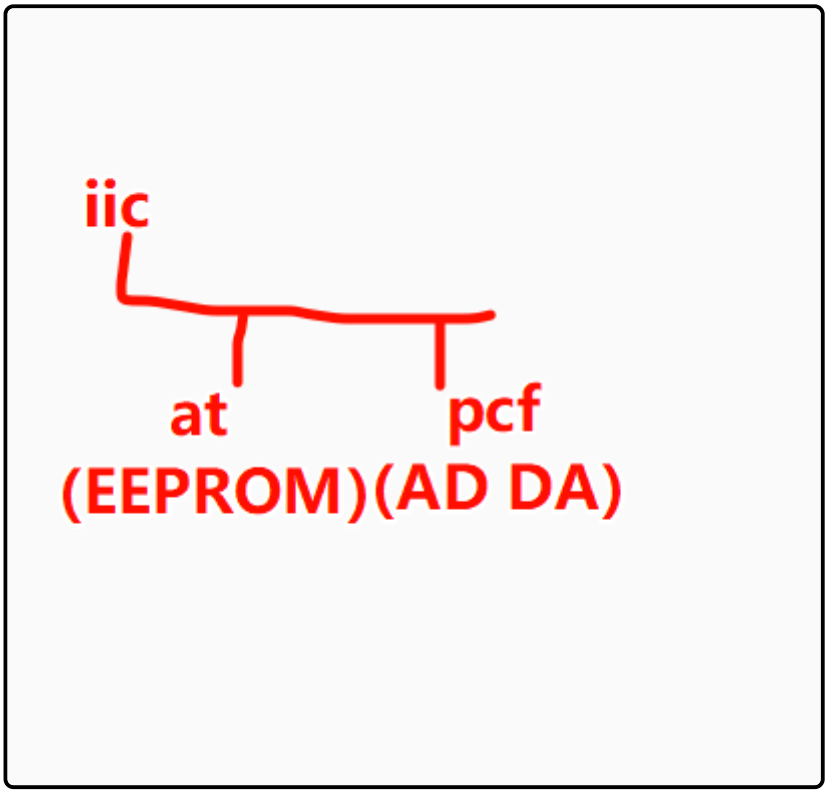
我们可以发现，底层中有scl，sda两个线没有定义，并且里面用到了nop函数，所以我们需要自己定义一下引脚和引入nop的头文件



```c
#include "intrins.h"
sbit scl = P2 ^ 0;
sbit sda = P2 ^ 1;
```

自定义的读写函数

在IIC通信中，所有外设都挂在总线上，通过地址的最后一位（0为写，1为读）来决定操作类型。我们需要记住ADDA是0x90（1001 0000），它挂在总线的第三个位置（而旁边的EEPROM的设备地址是0xA0（1010 0000），挂在第二个位置）。



IIC通信过程回顾

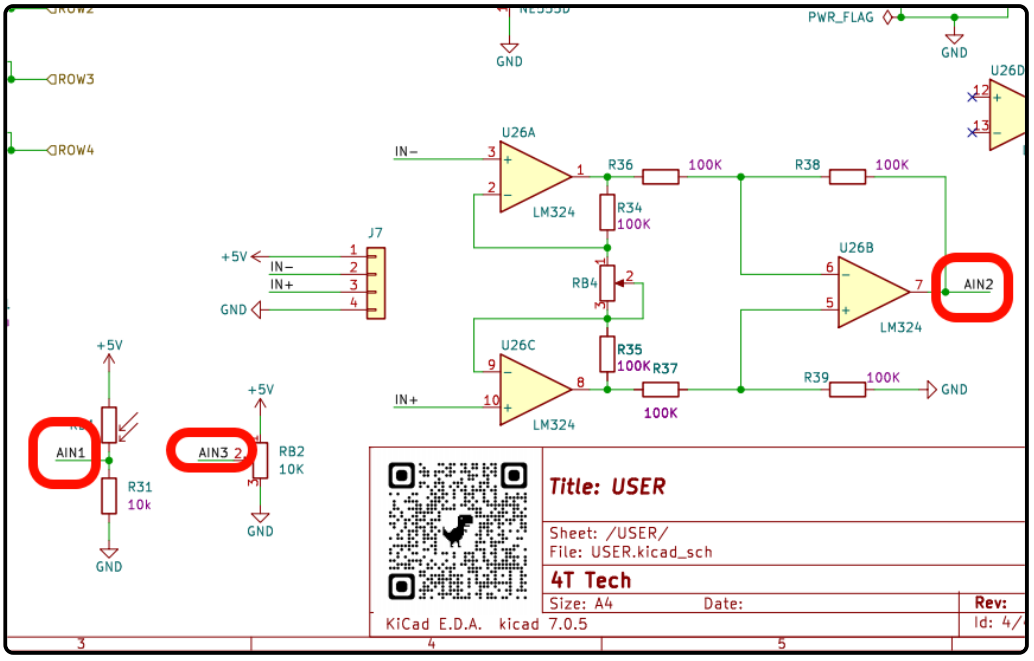
IIC通信基本流程：

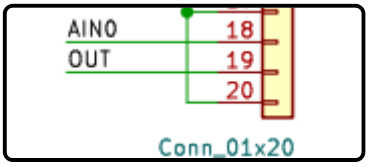
1. 选择芯片并等待回复
2. 进行写入或读取操作
3. 停止通信

我们知道，我们在这里主要是用的PCF8591芯片，那么我们对他进行操作的时候分为AD（读取）和DA（写入）

- **写入数据（DA）**：先发送0x90选择芯片，告诉他我们要对他开始进行操作了，然后开启DA的输出flag（0x4？），最后写入一下DA需要输出的数字电压（写入0-255映射为 0-5）
- **读取数据（AD）**：先发送0x90选择芯片，告诉他我们要对他开始进行操作了，然后写入对应的需要读取的参数（下面介绍），最后重新开启一次会话，读取结果就行了，读取到的结果是（0-255，映射为模拟电压0-5）

我们可以通过这个表格来看出我们需要凑出来的对应 的数据





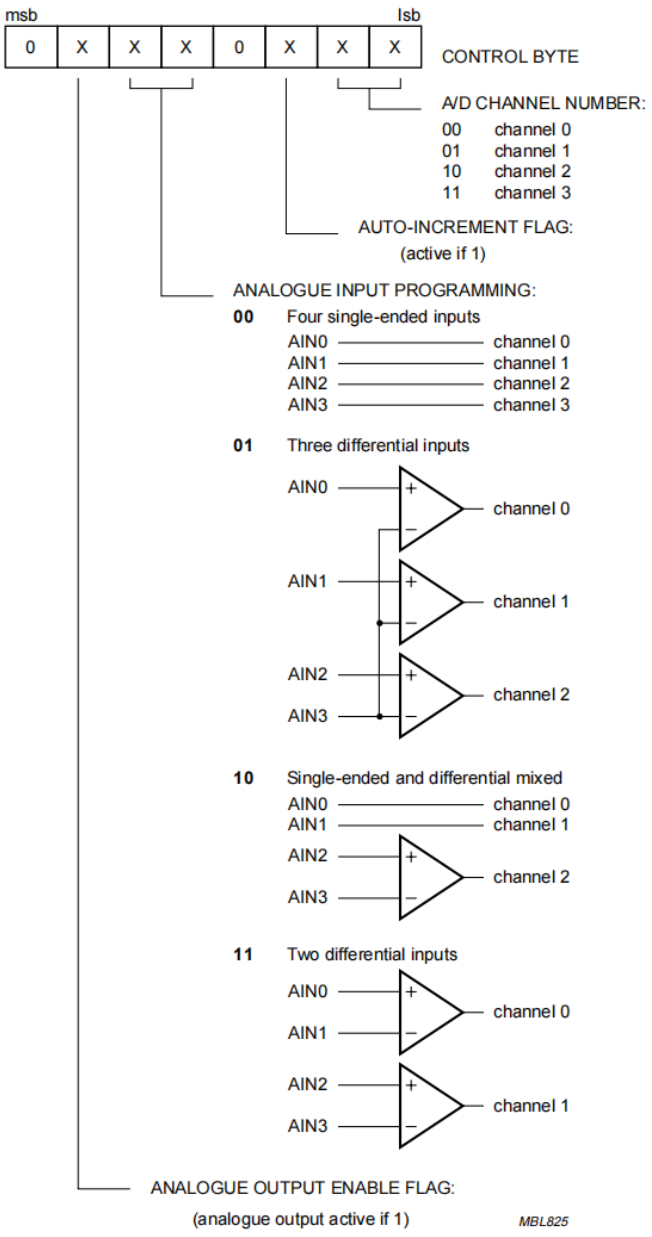


Fig.5 Control byte.

```
1
2   /**
3    * 功能: PCF8591 AD转换
4    * 描述:
5    *    - 从PCF8591的指定通道读取AD转换值
6    *    - 读取范围: 0~255 (对应0~5V)
7    * 参数:
8    *    addr - 控制字, 指定通道和配置
9    *      0x40/0x00 - 开启DA/不开启DA, 外部输入
10   *      0x41/0x01 - 开启DA/不开启DA, 光敏电阻输入
11   *      0x42/0x02 - 开启DA/不开启DA, 差分输入
12   *      0x43/0x03 - 开启DA/不开启DA, 滑动变阻器输入
```

```
 * 返回值:
 *   AD转换结果(0~255)
 */
unsigned char Ad_Read(unsigned char addr)
{
    unsigned char temp;

    I2CStart();                 // 发送起始信号
    I2CSendByte(0x90);          // 发送PCF8591设备地址+写操作(0x48<<1|0)
    I2CWaitAck();               // 等待应答
    I2CSendByte(addr);          // 发送控制字(通道选择)
    I2CWaitAck();               // 等待应答

    I2CStart();                 // 重新发送起始信号
    I2CSendByte(0x91);          // 发送PCF8591设备地址+读操作(0x48<<1|1)
    I2CWaitAck();               // 等待应答
    temp = I2CReceiveByte();    // 读取AD转换结果
    I2CSendAck(1);              // 发送非应答信号(结束读取)
    I2CStop();                  // 发送停止信号

    return temp;                // 返回AD转换结果
}

/**
 * 功能: PCF8591 DA转换
 * 描述:
 *   - 向PCF8591的DA输出设置模拟量
 *   - 设置范围: 0~255 (对应0~5V)
 * 参数:
 *   dat - DA转换值(0~255)
 */
void Da_Write(unsigned char dat)
{
    I2CStart();                 // 发送起始信号
    I2CSendByte(0x90);          // 发送PCF8591设备地址+写操作
    I2CWaitAck();               // 等待应答
    I2CSendByte(0x41);          // 发送控制字(开启DA输出)
    I2CWaitAck();               // 等待应答
    I2CSendByte(dat);           // 发送DA值
    I2CWaitAck();               // 等待应答
}
```

AD_DA应用层

我们在这里以v4T里面为例，首先，你不可能一直读取AD的数据吧，工作室测试的时间是80ms~160ms读取一次数据比较好，读取出来看情况转换为0-5V，看题目要求就行了。

```
void AD_DA()
{
    // 检查延时是否到达
    if (AD_Slow_Down < 80)
        return;
    AD_Slow_Down = 0;

    // 读取光照传感器AD值并转换为百分比(0-100)
    AD_Light_Val_100x = Ad_Read(0x41) * 100 / 51;

    // 读取RB2通道AD值并转换为百分比(0-100)
    AD_RB2_Val_100x = Ad_Read(0x43) * 100 / 51;

    // 设置DA输出为51*2=102
    Da_Write(2 * 51);
}

```

易错知识点

这里只有一个问题，我询问了4T官方，得到的答复是，以板子为准，所以我们在连续读取的时候，读取两次，或者干脆直接交换地址

工作室推荐交换地址进行读取，为什么会出现这个情况呢，因为我们PCF8591连读取到时候，下一次读取到的是上一次转换后的值（如果中间不间隔的情况下）。

```
// 连续读取两次
// 读取光照传感器AD值并转换为百分比(0-100)
Ad_Read(0x41);
AD_Light_Val_100x = Ad_Read(0x41) * 100 / 51;

// 读取RB2通道AD值并转换为百分比(0-100)
Ad_Read(0x43);
AD_RB2_Val_100x = Ad_Read(0x43) * 100 / 51;
```

```c
// 交换地址读取（反读）
// 读取光照传感器AD值并转换为百分比(0-100)
AD_Light_Val_100x = Ad_Read(0x43) * 100 / 51;

// 读取RB2通道AD值并转换为百分比(0-100)
AD_RB2_Val_100x = Ad_Read(0x41) * 100 / 51;
```